

FACULTAD DE INFORMÁTICA

Máster en Internet de las Cosas
2025-2026



Asignatura:

Diseño de Infraestructura Inteligente para IOT (DII)

Práctica:

Trabajos individuales grupales

Alumno:

Jose Villacres Zumaeta

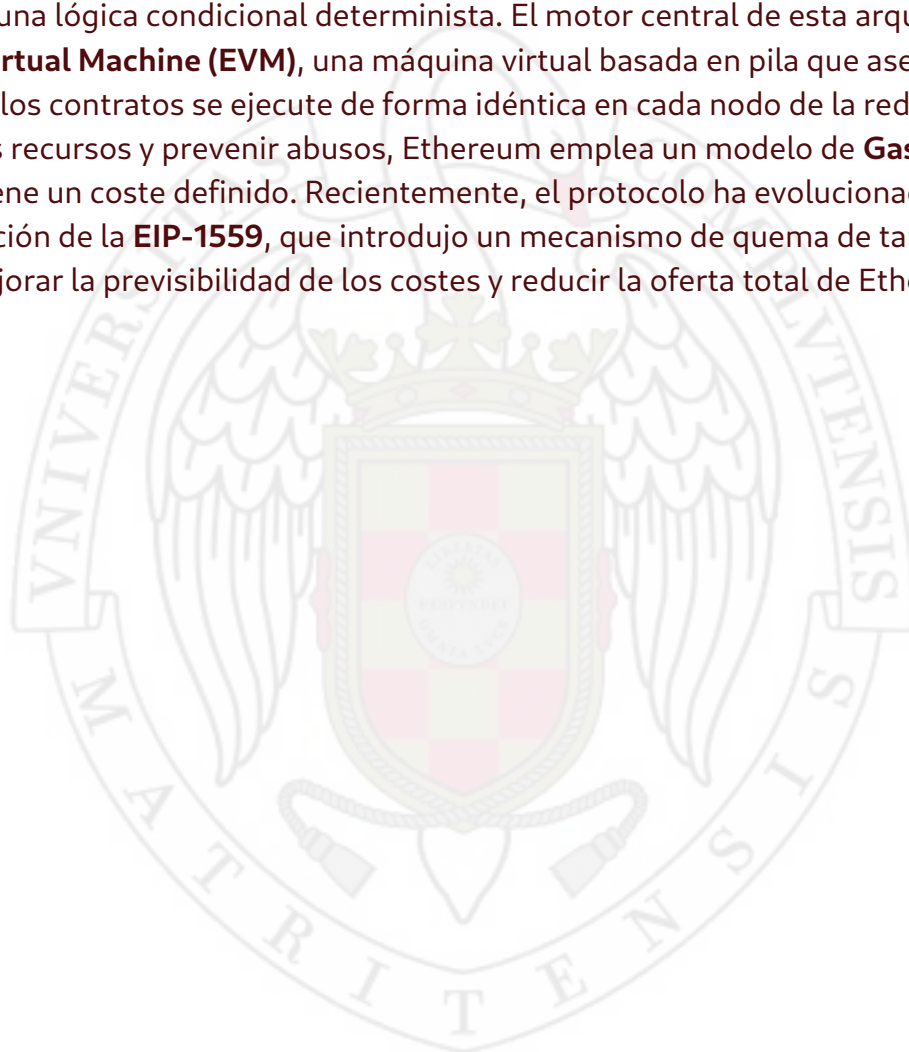
Sumario

Estado del Arte.....	3
Contratos inteligentes y la Ethereum Virtual Machine (EVM).....	4
Ethereum Virtual Machine (EVM).....	4
Modelo de gas y comisiones de ejecución.....	6
Usos y aplicaciones de los contratos inteligentes.....	7
Comparación con las capacidades de Bitcoin.....	8
Funcionamiento de los Smart Contracts en Solidity.....	9
El principio de “Código es Ley” (Code is Law).....	9
1.1 El lenguaje Solidity como sistema formal.....	9
1.2 Lógica condicional determinista.....	10
Auto-ejecución e inmutabilidad.....	10
2.1 Inmutabilidad del código.....	10
2.2 Ejecución automática sin intermediarios.....	10
Comparación entre contratos tradicionales y smart contracts.....	11
Analogía explicativa: la máquina expendedora.....	11
Anatomía de un Smart Contract en Solidity.....	12
5.1 Declaración de versión y definición del contrato.....	12
5.2 Variables de estado: la memoria persistente.....	12
5.3 Constructor y eventos.....	13
5.4 Funciones: lógica operativa y control de acceso.....	13
Síntesis conceptual para material didáctico.....	13
Conclusión.....	14



Estado del Arte

Actualmente, la red **Ethereum** se define como una "**computadora mundial**" distribuida, capaz de ejecutar cálculos replicados globalmente cuyos resultados son inmutables y transparentes. A diferencia de Bitcoin, que utiliza un lenguaje de scripting no Turing-completo para priorizar la simplicidad, Ethereum es la primera cadena de bloques en proporcionar un entorno **Turing-completo**. Este sistema permite la ejecución de **contratos inteligentes**, programas informáticos autoejecutables almacenados en la blockchain que operan bajo una lógica condicional determinista. El motor central de esta arquitectura es la **Ethereum Virtual Machine (EVM)**, una máquina virtual basada en pila que asegura que el bytecode de los contratos se ejecute de forma idéntica en cada nodo de la red. Para gestionar los recursos y prevenir abusos, Ethereum emplea un modelo de **Gas**, donde cada operación tiene un coste definido. Recientemente, el protocolo ha evolucionado con la implementación de la **EIP-1559**, que introdujo un mecanismo de quema de tarifas (base fee) para mejorar la previsibilidad de los costes y reducir la oferta total de Ether.



Contratos inteligentes y la Ethereum Virtual Machine (EVM)

Una de las innovaciones más relevantes introducidas por la red Ethereum es el concepto de **contratos inteligentes** (*smart contracts*), definidos como programas informáticos autoejecutables que se almacenan y ejecutan directamente sobre la blockchain. Estos contratos se ejecutan de manera **determinista y verificable** en todos los nodos participantes de la red, garantizando la coherencia global del estado del sistema distribuido.

Ethereum fue la primera plataforma blockchain en proporcionar un entorno **Turing-completo** sobre la cadena de bloques, lo que implica que, en términos teóricos, sus contratos pueden implementar cualquier función computacional general, sujeto a las limitaciones prácticas impuestas por los costos de ejecución y los límites de gas. Los contratos inteligentes se desarrollan habitualmente utilizando lenguajes de alto nivel como **Solidity** o **Vyper**, los cuales son posteriormente compilados a **bytecode** ejecutable por la Ethereum Virtual Machine (EVM).

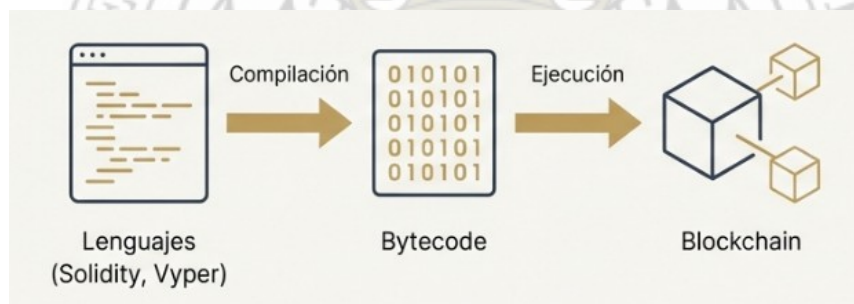


Figura 1: Programas informáticos auto-ejecutables.

Ethereum Virtual Machine (EVM)

La **Ethereum Virtual Machine (EVM)** constituye el entorno de ejecución estándar de los contratos inteligentes en Ethereum. Se trata de una **máquina virtual basada en pila** (*stack-based virtual machine*) diseñada para ejecutar el bytecode de los contratos de forma idéntica en cada nodo de la red. Cada nodo ejecuta una instancia local de la EVM, lo que permite reproducir exactamente las mismas computaciones a partir de una transacción de entrada y un estado previo determinado.

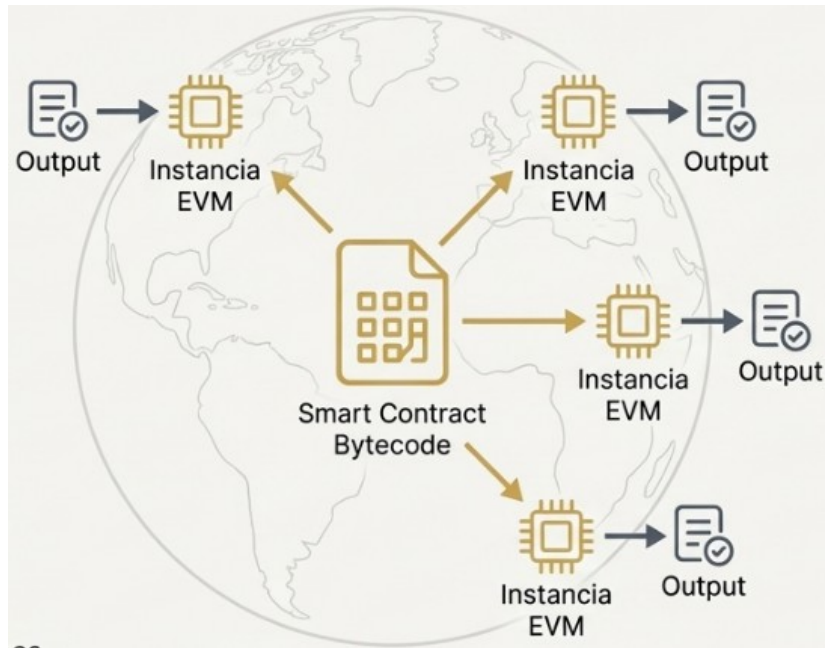


Figura 2: El motor de ejecución distribuido.

Este diseño asegura que Ethereum funcione como una **“computadora mundial” distribuida**, en la cual los cálculos se replican globalmente y los resultados —incluidos los cambios de estado— son inmutables, transparentes y verificables. La EVM define un conjunto formal de **opcodes** (instrucciones de bajo nivel) y un modelo de costos asociado, medido en unidades de **gas**, para cada operación computacional. Dicho esquema permite cuantificar de manera predecible el consumo de recursos.

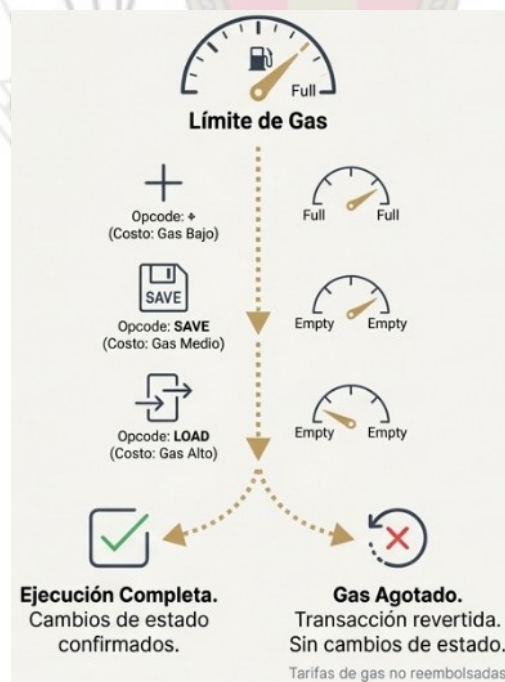


Figura 3: El rol del Gas en EVM

Con el fin de prevenir bucles infinitos y el abuso de recursos computacionales, cada transacción debe especificar un **límite máximo de gas**. En caso de que la ejecución de un contrato consuma todo el gas disponible antes de finalizar, la transacción se revierte y no se aplican cambios al estado global, aunque el gas consumido no es reembolsado. Este mecanismo incentiva el desarrollo de contratos eficientes y seguros.

Modelo de gas y comisiones de ejecución

Cada operación ejecutada por la EVM —como operaciones aritméticas, llamadas a funciones, acceso a memoria o escritura en almacenamiento persistente— consume una cantidad fija y previamente definida de gas. El gas actúa como una unidad abstracta que mide el esfuerzo computacional, mientras que las comisiones asociadas se pagan en **Ether (ETH)** en función del gas utilizado.

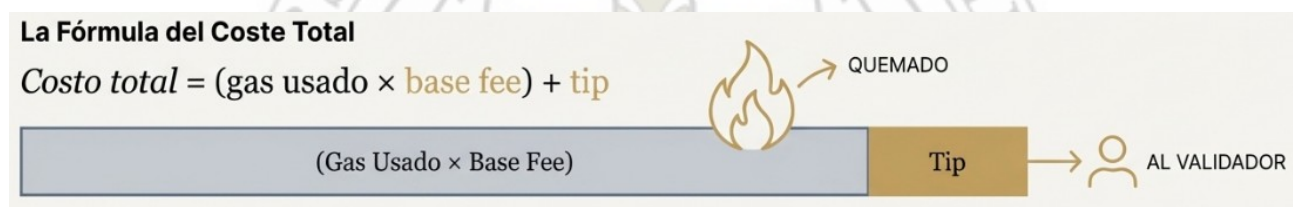


Figura 4: Impacto económico.

Antes de la implementación de la propuesta **EIP-1559**, Ethereum utilizaba un modelo de subasta de primer precio, en el que los usuarios especificaban un *gas price* y los mineros priorizaban las transacciones más rentables. Sin embargo, desde 2021, EIP-1559 introdujo un **modelo híbrido de tarifas**, compuesto por una **tarifa base (base fee)** ajustada automáticamente por el protocolo según la congestión de la red, y una **propina (tip)** opcional destinada al validador que incluye la transacción en un bloque.

Un aspecto clave de este modelo es que la **tarifa base se quema**, es decir, se elimina permanentemente de la circulación, reduciendo la oferta total de ETH. De este modo, el costo total de una transacción se calcula como:

Este mecanismo contribuye a estabilizar las comisiones, mejora la previsibilidad para los usuarios y establece una relación directa entre la actividad de la red y la reducción de la oferta monetaria de Ether.

Usos y aplicaciones de los contratos inteligentes

La capacidad de implementar lógica arbitraria y persistente ha dado lugar a un amplio ecosistema de **aplicaciones descentralizadas (DApps)** que abarcan múltiples dominios:



Figura 5: Más allá de la teoría.

Finanzas descentralizadas (DeFi): Protocolos para préstamos, intercambios descentralizados (DEX), mercados monetarios, emisión de activos sintéticos y *staking* líquido, operando sin intermediarios centralizados (por ejemplo, Uniswap, Aave y Compound).

- **Tokens y activos digitales:** Ethereum introdujo estándares ampliamente adoptados como **ERC-20** para tokens fungibles y **ERC-721** para tokens no fungibles (NFT), facilitando fenómenos como las ICO y el auge del arte digital y los coleccionables.
- **Organizaciones Autónomas Descentralizadas (DAO):** Sistemas de gobernanza y gestión de fondos codificados en contratos inteligentes, que permiten la coordinación colectiva de recursos de manera transparente.
- **Identidad digital y registros inmutables:** Aplicaciones orientadas a la gestión de identidades, certificados académicos, títulos de propiedad y trazabilidad en cadenas de suministro.
- **Juegos y metaverso:** Plataformas que integran economías digitales basadas en NFT y mundos virtuales descentralizados, como Decentraland.

En conjunto, estos ejemplos ilustran la versatilidad de Ethereum como plataforma de propósito general, limitada únicamente por los costos de gas y el rendimiento de la red.

Comparación con las capacidades de Bitcoin

Bitcoin fue diseñado deliberadamente con un lenguaje de scripting **no Turing-completo**, priorizando la simplicidad y la seguridad del sistema. Sus scripts permiten condiciones básicas —como pagos multifirma, *timelocks* o mecanismos criptográficos utilizados en Lightning Network— pero no soportan lógica compleja ni almacenamiento de estado persistente.

En Bitcoin, los scripts se ejecutan únicamente durante la validación de transacciones y no existen programas persistentes en la cadena. Como consecuencia, Bitcoin no admite aplicaciones descentralizadas complejas en su capa base. Ethereum, en contraste, fue concebido desde su origen como una plataforma programable de propósito general, donde los contratos pueden mantener estado, interactuar entre sí y componer funcionalidades complejas.

Si bien existen iniciativas que buscan ampliar la funcionalidad de Bitcoin —como cadenas laterales compatibles con EVM (por ejemplo, Rootstock) o soluciones de capas superiores — estas aproximaciones no forman parte del diseño fundamental del protocolo, sino que constituyen extensiones externas.

Característica	Bitcoin	Ethereum
Propósito Principal	Transferencia de valor simple y segura	Plataforma programable de propósito general
Lenguaje de Scripting	No Turing-completo	Turing-completo
Capacidades del Script	Lógica básica: multifirma, timelocks	Lógica compleja y arbitraria
Estado	No soporta estado persistente en la cadena	Los contratos mantienen su propio estado persistente
Modelo de Cuenta	UTXO (Salidas de Transacción no Gastadas)	Modelo de Cuentas (Externas y de Contrato)
Aplicaciones Nativas	No admite DApps complejas en su capa base	Diseñado para un ecosistema de DApps

Figura 6: Simplicidad y la seguridad contra la programabilidad y flexibilidad

Funcionamiento de los Smart Contracts en Solidity

Los *smart contracts* constituyen uno de los pilares tecnológicos de las redes blockchain programables, particularmente de Ethereum. A diferencia de los contratos tradicionales, cuya interpretación y ejecución dependen de intermediarios humanos y marcos jurídicos flexibles, los *smart contracts* se implementan como programas informáticos que definen reglas estrictas de ejecución automática. Estos contratos están escritos en lenguajes formales, siendo **Solidity** el más ampliamente utilizado en el ecosistema Ethereum.

El objetivo de esta sección es exponer, de manera sistemática y accesible, los principios lógicos, técnicos y estructurales que rigen el funcionamiento de los *smart contracts*, incluso para audiencias no especializadas en programación.

El principio de “Código es Ley” (Code is Law)



Figura 7: Código es Ley

En los sistemas contractuales tradicionales, el significado de un acuerdo puede ser objeto de interpretación por jueces o árbitros. En contraste, dentro de una blockchain, el **código fuente del contrato constituye la autoridad última**. No existe margen para ambigüedad semántica ni reinterpretación posterior.

1.1 El lenguaje Solidity como sistema formal

Solidity puede entenderse como el “idioma” que la máquina virtual de Ethereum (EVM) es capaz de interpretar y ejecutar. Se trata de un lenguaje:

- Determinista
- Estrictamente tipado
- Basado en reglas lógicas explícitas

Cada instrucción debe estar definida con precisión matemática, eliminando cualquier posibilidad de interpretación subjetiva.

1.2 Lógica condicional determinista

La lógica de un *smart contract* se articula fundamentalmente mediante estructuras condicionales del tipo:

Si se cumple una condición A, entonces se ejecuta una acción B.

No existen estados intermedios, excepciones informales ni juicios de valor. Por ejemplo:

Si el contrato recibe exactamente 2 ETH, entonces transfiere la propiedad del NFT #402 al remitente.

Este enfoque garantiza previsibilidad total del comportamiento del sistema.

Auto-ejecución e inmutabilidad

El funcionamiento de los *smart contracts* depende directamente de la infraestructura descentralizada de Ethereum, la cual aporta dos propiedades fundamentales:

inmutabilidad y ejecución automática.

2.1 Inmutabilidad del código

Una vez que un contrato en Solidity es desplegado en la blockchain:

- Su código no puede ser modificado.
- Las reglas quedan permanentemente registradas.
- Ningún participante puede alterarlas unilateralmente.

Esta característica elimina la necesidad de confianza interpersonal y la sustituye por confianza en el protocolo y en la verificación criptográfica.

2.2 Ejecución automática sin intermediarios

El *smart contract* opera como un agente autónomo e imparcial. Permanece inactivo hasta que las condiciones predefinidas se cumplen, momento en el cual ejecuta automáticamente las acciones correspondientes, tales como:

- Liberación de fondos retenidos en garantía.
- Transferencia de activos digitales o tokens.
- Concesión de acceso a servicios o sistemas digitales.

Todo ello ocurre sin la intervención de abogados, notarios, bancos u otras entidades centralizadas.

Comparación entre contratos tradicionales y smart contracts

Característica	Contrato Tradicional	Smart Contract (Solidity)
Soporte	Papel / PDF	Código en blockchain
Ejecución	Manual, dependiente de humanos	Automática e inmediata
Intermediarios	Abogados, notarios, bancos	Ninguno
Seguridad	Vulnerable a pérdida o alteración	Inmutable, protegida criptográficamente

Analogía explicativa: la máquina expendedora

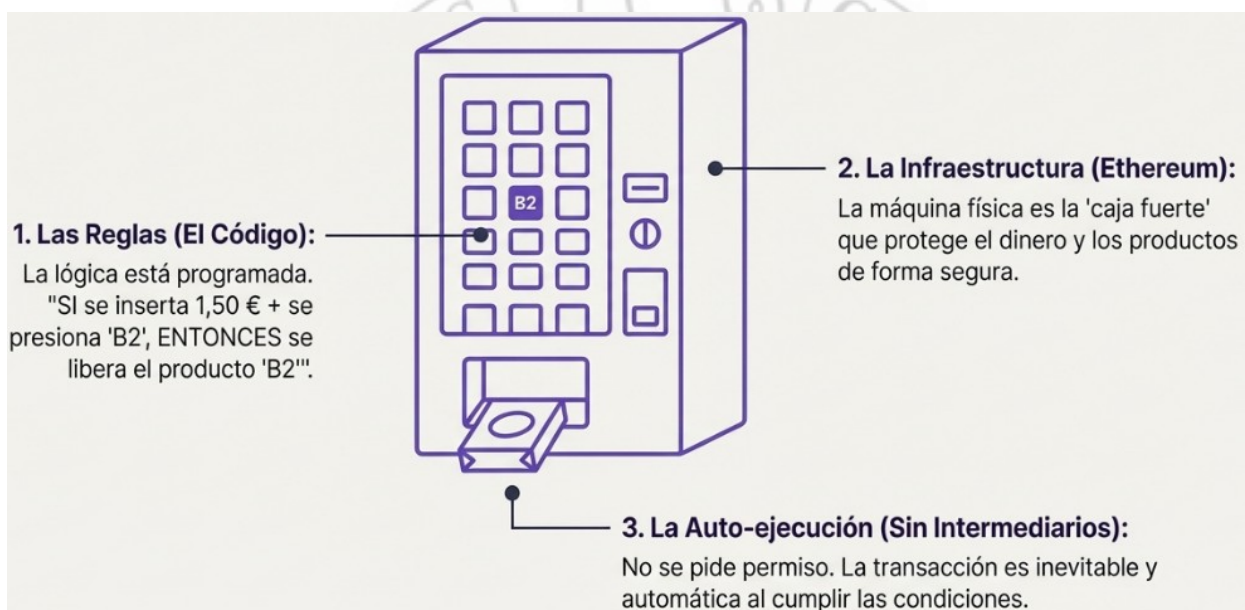


Figura 8: La famosa máquina expendedora.

Una analogía pedagógica eficaz para comprender los *smart contracts* es la máquina expendedora:

- Reglas (Código):**
La máquina está programada para ejecutar una acción específica ante una entrada determinada.
- Infraestructura (Ethereum):**
La máquina protege tanto el producto como el dinero mediante mecanismos físicos, de forma análoga a la seguridad criptográfica de la blockchain.
- Auto-ejecución:**
No se requiere autorización humana; al introducir la moneda correcta, el resultado es inevitable.

Esta analogía ilustra claramente la naturaleza determinista y automática de los *smart contracts*.

Anatomía de un Smart Contract en Solidity

Desde una perspectiva estructural, un contrato en Solidity puede analizarse como el diseño arquitectónico de una “caja fuerte digital”. El archivo `.sol` presenta una jerarquía lógica bien definida, compuesta por cuatro elementos fundamentales.

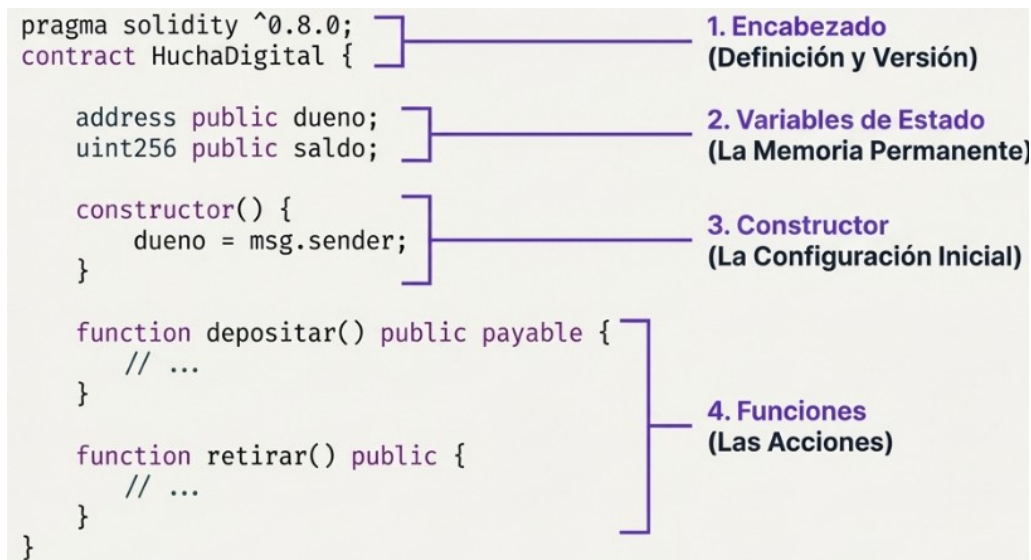


Figura 9: La anatomía del Smart Contract

5.1 Declaración de versión y definición del contrato

Al igual que un documento legal especifica su marco normativo, un contrato en Solidity debe declarar la versión del compilador utilizada, con el fin de evitar inconsistencias semánticas.

```
pragma solidity ^0.8.0;

contract HuchaDigital {
    // Definición del contrato
}
```

5.2 Variables de estado: la memoria persistente

Las variables de estado almacenan información directamente en la blockchain y presentan las siguientes características:

- Persistencia a largo plazo.
- Coste económico asociado (Gas).
- Replicación en todos los nodos de la red.

```
address public dueno;
uint256 public saldo;
```

5.3 Constructor y eventos

Constructor

Función especial que se ejecuta una única vez durante el despliegue del contrato y establece las condiciones iniciales.

Eventos

Mecanismos de notificación que permiten al contrato comunicar sucesos relevantes a aplicaciones externas.

```
event DepositoRealizado(address quien, uint cantidad);

constructor() {
    dueno = msg.sender;
    saldo = 0;
}
```

5.4 Funciones: lógica operativa y control de acceso

Las funciones representan las acciones que los usuarios pueden ejecutar. Se distinguen principalmente dos tipos:

- **Funciones de lectura:** no modifican el estado y no consumen Gas.
- **Funciones de escritura:** alteran el estado y requieren Gas.

Asimismo, incorporan mecanismos de seguridad mediante condiciones explícitas.

```
function depositar() public payable {
    saldo += msg.value;
    emit DepositoRealizado(msg.sender, msg.value);
}

function retirar() public {
    require(msg.sender == dueno, "No autorizado");
    payable(dueno).transfer(address(this).balance);
}
```

Síntesis conceptual para material didáctico

El funcionamiento de un *smart contract* puede resumirse mediante tres componentes esenciales:

1. **La lógica (Funciones):** define qué acciones son posibles.
2. **El estado (Variables):** almacena datos y activos.
3. **La seguridad (Requisitos):** impone restricciones que previenen usos indebidos.

Conclusión

Ethereum representa una evolución sustancial respecto al diseño original de Bitcoin al consolidarse como una **plataforma programable de propósito general** que utiliza un modelo de consenso moderno basado en **Proof of Stake (PoS)**. El análisis de su estructura revela que la combinación de la **EVM**, la inmutabilidad del código y la ejecución automática sin intermediarios permite el desarrollo de un ecosistema diverso de **aplicaciones descentralizadas (DApps)**, que abarcan desde las finanzas descentralizadas (DeFi) y tokens (ERC-20, ERC-721) hasta organizaciones autónomas (DAO) y registros de identidad inmutables. Bajo el principio de **"Código es Ley"** (Code is Law), los contratos inteligentes eliminan la ambigüedad semántica de los acuerdos tradicionales, garantizando que las reglas se cumplan de manera estrictamente matemática. Finalmente, aunque la red enfrenta desafíos de **escalabilidad**, el desarrollo de soluciones de segunda capa y actualizaciones progresivas del protocolo aseguran su posición dominante en la infraestructura inteligente para el Internet de las Cosas y la economía digital.



Bibliografía

- VanEck. ***Bitcoin vs. Ethereum in 2025: Comparison & Outlook***. VanEck Peru.
- Ethereum Official Documentation – Proof-of-Stake Consensus
- Sentinel Assam News (2025) – Ethereum Staking in 2025
- Ethereum Improvement Proposal 1559 – Fee market change for ETH 1.0 chain (2021).
- Ethereum Whitepaper – V. Buterin, “Ethereum: A Next-Generation Smart Contract and Decentralized
- Application Platform” (2014).

